

**Contrôle de connaissances,
Programmation orientée objet C++
Durée 1h, aucun document n'est autorisé**

Une date est donnée par le jour, le mois, et l'année tous de type entiers. La classe correspondante admet un constructeur prenant en arguments 3 valeurs (par défaut (0,0,0)).

1-a Écrire la classe date avec son constructeur

1-b Écrire une fonction de comparaison :

```
if(d1.inferieure(d2)) { ..... }
```

retourne vrai si d1 est antérieure à d2 (ou égale)

1-c-a Écrire une fonction qui prend en argument un tableau de dates et sa taille et qui retourne la plus petite et la plus grande date (selon l'ordre défini par la fonction inférieure)

1-c-b Compléter le main suivant pour montrer que votre fonction du 1-c-a marche :

```
int main()
{
    int N=100 ;
    // La fonction initialiser_alea cree un tableau de N dates, l'
    // initialise aleatoirement
    date *T=initialiser_alea(T,N) ;

    // A ce stade T est un tableau de N dates
    // completez ....
    ...
}
```

1-d-a Écrire la fonction *nb_anterieures* qui prend en argument un tableau de dates, sa taille et une date D. La fonction renvoie le nombre de dates du tableau qui sont antérieures à D.

1-d-b Écrire la fonction *anterieures* qui prend en argument un tableau de dates, sa taille et une date D. La fonction renvoie un tableau de dates (alloué dynamiquement) contenant toutes les dates du tableau qui sont antérieures à D. La fonction renvoie NULL si aucune date n'est antérieure à D. N'oublier pas de retourner la taille du nouveau tableau.

1-d-c Compléter le main suivant pour montrer que votre fonction du 1-d-b marche :

```
int main()
{
    int N=100 ;
    // La fonction initialiser_alea cree un tableau de N dates, l'
    // initialise aleatoirement
    date *T=initialiser_alea(T,N) ;

    // A ce stade T est un tableau de N dates
    // completez pour recuperer le tableau R contenant toutes les date
    // de T <= 01/12/2000
    ....
    // afficher le tableau R
}
```

Nous souhaitons stocker les dates dans une liste chaînée. Pour cela nous définissons la classe *Maillon* :

```
class Maillon
{
    friend class liste_dates ;
```

```

        date D ;
        Maillon *suivant ;
    public :
        Maillon(int j=1,int m=1 , a=2000){ (*this).D.j=1 ; (*this).D.m=m ;
            (*this).D.a=a;}
        ...
} ;

```

Soit la classe *liste_dates* :

```

class liste_dates
{
    maillon *tete ;
    public :
        liste_dates(){ (*this).tete=NULL;}
        ...
}

```

2-b Écrire la fonction qui insère en tête une date :

```
L.inserer_tete(int j, int m, int a) ; // L est une liste chainee de
    dates
```

2-c-a Écrire une fonction *nb_dates_anterieures* de la classe *liste_dates* qui prend en argument une date D et qui retourne le nombre de dates de la liste qui sont antérieures à D.

2-c-b Écrire la fonction *anterieures* de la classe *liste_dates* qui prend en argument une date D. La fonction renvoie un tableau de dates (alloué dynamiquement) contenant toutes les dates de la liste qui sont antérieures à D. La fonction renvoie NULL si aucune date n'est antérieure à D. N'oublier pas de retourner la taille du nouveau tableau (utiliser 2-c-a pour compter avant d'allouer).

3-a Soit la classe *arbre_dates* permettant de stocker des dates :

```
class Noeud
{
    friend class arbre_dates ;
    date D ;
    Noeud *fg ;
    Noeud *fd ;
public :
    ...
} ;

class arbre_dates
{
    Noeud *racine ;
public :
    arbre_dates(){racine=NULL;}
    ...
} ;
```

3-c-a Écrire une fonction récursive *nb_dates_anterieures* de la classe *arbre_dates* qui prend en argument une date D et qui retourne le nombre de dates de l'arbre qui sont antérieures à D.

3-c-b Écrire la fonction *anterieures* de la classe *arbre_dates* qui prend en argument une date D. La fonction renvoie un tableau de dates (alloué dynamiquement) contenant toutes les dates de l'arbre qui sont antérieures à D. La fonction renvoie NULL si aucune date n'est antérieure à D. N'oublier pas de retourner la taille du nouveau tableau (utiliser 3-c-a pour compter avant d'allouer).